



CYNA

Document d'Architecture Technique (DAT)

Projet fil rouge — Plateforme Cyna

**Adrien CACHOUX · Romain CANER ·
Ethan MIRBEAU**

Version 2.0 — Bloc 3

Solutions de cybersécurité SaaS — SOC · EDR · XDR

Positionnement du document. Le DAT est le blueprint technique de la solution : il décrit l'architecture **telle que réalisée et exploitée** (composants, versions, flux, dimensionnement, sécurité, exploitation) et justifie les choix. Il se distingue :

- du **DCG** (conception générale), qui reste au niveau fonctionnel ;
- du **DCT** (conception technique), qui détaille le modèle de données et les choix d'ingénierie de développement.

Le DAT agrège et renvoie vers les livrables d'exploitation détaillés (CI/CD, supervision, performance, incidents).

Table des matières

1. Vue d'ensemble
 2. Architecture applicative
 3. Architecture infrastructure
 4. Architecture des données
 5. Sécurité
 6. DevOps & exploitation
 7. Répartition rédactionnelle par candidat
 8. Matrice de traçabilité des livrables Bloc 3
 9. Annexes & références
-

1. Vue d'ensemble

1.1 Contexte projet

Cyna commercialise des solutions de cybersécurité SaaS (SOC, EDR, XDR). La plateforme livrée dote l'entreprise d'un canal de vente en ligne et d'un outil de gestion interne. Elle comprend :

- un **site web e-commerce mobile-first** (catalogue, recherche à facettes, panier, tunnel de commande, compte client, historique et factures) ;
- un **back-office web** (gestion produits/commandes/utilisateurs, tableaux de bord de ventes, 2FA obligatoire) ;
- une **application d'administration de bureau** (Java Swing) partageant la même base de données.

Périmètre. L'application mobile native est remplacée par un site **responsive mobile-first** et le **chatbot est hors périmètre** (arbitrage MoSCoW — voir [BILAN_PROJET.md](#) §2). Le formulaire de contact classique est livré.

1.2 Principes architecturaux structurants

Principe	Décision	Justification
Style applicatif	Monolithe modulaire (MVC en couches), pas de microservices	Taille du projet, équipe de 3, déploiement simple, latence minimale
Dépendances runtime	Aucune dépendance externe côté web (PHP natif)	Green IT, surface d'attaque réduite, maîtrise totale du noyau
Reproductibilité	Conteneurisation Docker identique dev → prod	Élimine les écarts d'environnement
Partage de données	Base MySQL unique web ↔ application Java	Cohérence, source de vérité unique
Coût d'hébergement	VPS gratuit à vie (Oracle Always Free)	Contrainte budgétaire du projet école

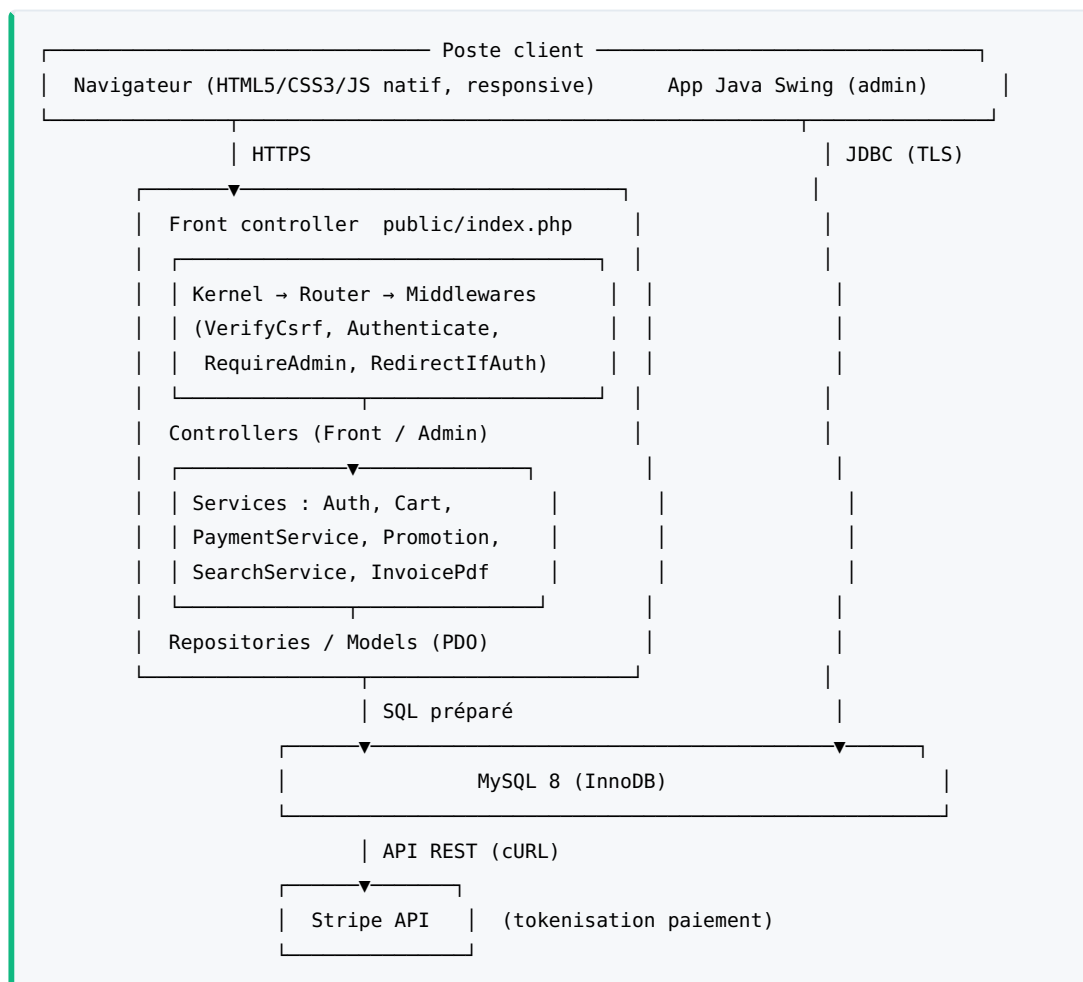
1.3 Contraintes techniques (exigences non fonctionnelles)

Contrainte	Cible	Source
Performance — pages clés	Temps de réponse < 2 s	Note de cadrage §4.4

Contrainte	Cible	Source
Performance — recherche	Résultats < 100 ms	CDC §VIII
Sécurité	Chiffrement, protection SQLi/XSS/CSRF, SSL, 2FA	CDC §XVIII
Scalabilité	Application sans état, montée en charge horizontale possible	Note de cadrage §9
Accessibilité	WCAG 2.1 AA	CDC §XVIII
Conformité	RGPD (données personnelles & paiement)	CDC §XIII

2. Architecture applicative

2.1 Diagramme de composants



2.2 Technologies utilisées (avec versions)

Pôle web (site + back-office)

Composant	Technologie / Version	Rôle
Langage	PHP 8.2 (natif, sans framework)	Logique applicative
Serveur	Apache 2.4 (image <code>php:8.2-apache</code>), <code>mod_rewrite</code>	Serveur HTTP, réécriture d'URL
Front-end	HTML5 / CSS3 / JavaScript natif + design system maison	Interface responsive mobile-first
Autoload	PSR-4 maison (namespace <code>Cyna\</code>)	Chargement des classes
Tests	PHPUnit 11.5 (<code>require-dev</code>)	Tests unitaires
Gestion dépendances	Composer (dev uniquement)	Outillage, aucun paquet en production

Pôle application d'administration (client lourd)

Composant	Technologie / Version	Rôle
Langage/UI	Java 17 + Swing	Client de bureau administrateur
Look & feel	FlatLaf 3.4.1	Thème moderne de l'interface Swing
Graphiques	JFreeChart 1.5.4	Tableaux de bord de ventes
Hachage	bcrypt 0.4	Vérification des mots de passe (bcrypt)
2FA	googleauth 1.5.0	TOTP (RFC 6238)
Accès BDD	mysql-connector-j 8.3.0	Pilote JDBC MySQL
Tests	JUnit 5.11.3 (scope test)	Tests unitaires
Packaging	Fat JAR (Maven Shade)	Distribution <code>java -jar</code> autonome

Services externes

Service	Usage	Intégration
Stripe	Paiement, tokenisation (PCI-DSS)	API REST via cURL, sans SDK ; repli simulé sans clés
SMTP	E-mails transactionnels	Client SMTP maison (Mailpit en dev)

2.3 Patterns d'architecture

- **Front Controller** : point d'entrée unique `public/index.php` → `Kernel`.
- **MVC en couches** avec séparation stricte : `Controllers` → `Services` → `Repositories` → `Models`, rendu par `View` (gabarits PHP, héritage de layout).
- **Middleware pipeline** : `VerifyCsrf`, `Authenticate`, `RequireAdmin`, `RedirectIfAuthenticated` exécutés avant l'action.
- **Repository pattern** : requêtes SQL préparées isolées par ressource.

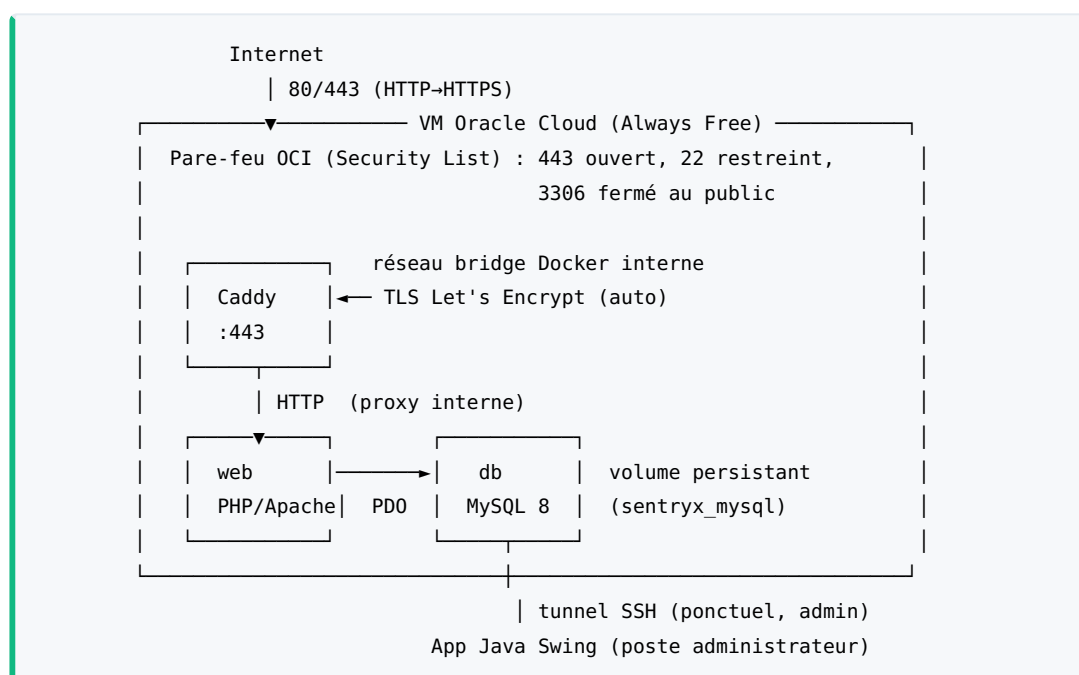
- **Service layer** : logique métier (panier, paiement, recherche, promotions, génération de factures PDF) découplée des contrôleurs.
- **DAO / JDBC** côté Java pour l'accès partagé à la même base.

2.4 API et interfaces

- **Interface web (HTTP/HTML)** : routes définies dans `routes/web.php` (front-office) et `routes/admin.php` (back-office). Verbes `GET` / `POST`, protection CSRF sur toutes les mutations.
- **Endpoint de données du tableau de bord** : `GET /admin/statistiques` renvoie du **JSON** (histogrammes, camembert) consommé par le front du back-office.
- **Contrat d'API documenté** : `api/openapi.yaml` (OpenAPI 3.0, exploitable dans Swagger UI / Postman).
- **Interface externe sortante** : appels REST vers l'API Stripe.
- **Interface Java ↔ BDD** : accès JDBC direct à MySQL (même schéma que le web).

3. Architecture infrastructure

3.1 Topologie & réseau



L'accès de l'application Java à MySQL se fait via **tunnel SSH** (le port 3306 n'est jamais exposé publiquement).

3.2 Serveurs et dimensionnement

Élément	Choix retenu	Dimensionnement
Hébergeur	Oracle Cloud Infrastructure — palier Always Free	Coût 0 € à vie
Machine (cible)	VM ARM Ampere A1 (si dispo)	jusqu'à 4 OCPU / 24 Go RAM (quota gratuit)
Machine (repli)	VM AMD E2.1.Micro	1 OCPU / 1 Go RAM + swap 2 Go
OS	Ubuntu 22.04 LTS	—
Stockage bloc	Volume Always Free	jusqu'à 200 Go
Nom de domaine	DuckDNS (sous-domaine gratuit)	—

Le dimensionnement modeste est cohérent avec la démarche **Green IT** : runtime web sans dépendance, images WebP, pagination, cache statique (objectif < 2 s).

3.3 Hébergement cloud & services utilisés

- **OCI Compute** (VM) pour l'exécution.
- **OCI Security Lists** (équivalent Security Groups) pour le filtrage réseau.
- **Let's Encrypt** via Caddy pour les certificats TLS.
- Pas de service managé (base, file d'attente) : tout est conteneurisé sur la VM pour rester dans le palier gratuit. Piste d'évolution : base managée + load balancer pour une vraie haute disponibilité.

3.4 Conteneurisation & orchestration

- **Docker + Docker Compose** ; images : `php:8.2-apache` (construite via `Dockerfile`), `mysql:8.0`, `caddy` (prod), `axllent/mailpit` et `phpmyadmin` (dev uniquement, non exposés en production).

- **Orchestration** : Docker Compose (pas de Kubernetes — surdimensionné pour une VM unique). Le passage à Kubernetes/Swarm est une piste d'évolution documentée si une montée en charge horizontale devient nécessaire.
 - **Rotation des logs** conteneurs configurée (`json-file` , `max-size 10m` , `max-file 5`) — voir [SUPERVISION_LOGS.md](#).
-

4. Architecture des données

4.1 Base de données

- **SGBD** : **MySQL 8.0**, moteur **InnoDB** (transactions ACID indispensables au tunnel de commande), jeu de caractères `utf8mb4_unicode_ci`.
- **Schéma** (`Cyna/database/schema.sql`) — 14 tables :

Domaine	Tables
Catalogue	<code>categories</code> , <code>produits</code> , <code>images_produit</code>
Comptes & facturation	<code>utilisateurs</code> , <code>adresses</code> , <code>moyens_paiement</code>
Commandes & abonnements	<code>commandes</code> , <code>lignes_commande</code> , <code>abonnements</code>
Contenu & réglages	<code>diapositives_accueil</code> , <code>reglages</code> , <code>codes_reduction</code>
Support & licences	<code>messages_contact</code> , <code>installations</code>

- **Indexation** : index sur les colonnes de tri/filtre (ex. tri des produits, date de commande) pour respecter l'objectif de recherche < 100 ms.
- **Accès** : exclusivement par **requêtes préparées PDO** (web) et **PreparedStatement JDBC** (Java) — aucune concaténation de requête.

4.2 Flux de données

Flux	Description
Commande client	Panier (session) → checkout → transaction (commande + lignes + abonnements) → paiement Stripe → e-mail de confirmation
Paiement	Données carte tokenisées côté Stripe ; seul un identifiant/token et les 4 derniers chiffres transitent et sont stockés
Administration	Application Java (JDBC) et back-office web lisent/écrivent la même base (source de vérité unique)

Flux	Description
Statistiques	Agrégations SQL → JSON <code>GET /admin/statistiques</code> → graphiques JFreeChart / JS
E-mails	Application → SMTP (confirmation d'inscription, réinitialisation, commande)

4.3 Stockage

- **Données relationnelles** : volume Docker persistant `sentryx_mysql` (`/var/lib/mysql`).
- **Fichiers** : images produits et assets servis depuis le conteneur web (dossier `public/`), images en **WebP**.
- **Factures** : générées à la volée en **PDF** (`Services/InvoicePdf`) — pas de stockage binaire redondant.

4.4 Sauvegarde et restauration (RPO / RTO)

Paramètre	Valeur	Détail
Méthode	<code>mysqldump</code> quotidien compressé (cron 02h30)	<code>INFRA_SAUVEGARDE_SSL.md</code> + <code>scripts/backup-db.sh</code>
Rétention	14 jours glissants	Purge automatique
RPO (perte de données max.)	≤ 24 h	Fréquence quotidienne ; réductible via sauvegardes plus fréquentes / binlog
RTO (temps de remise en service)	~ 1 à 2 h	Redéploiement Docker + restauration du dump
Résilience	Copie hors-site recommandée (objet/S3, autre VPS) + chiffrement des archives (RGPD)	Piste d'amélioration

Bonne pratique : tester régulièrement une restauration (un backup jamais restauré n'est pas un backup).

5. Sécurité

La sécurité est traitée by design à chaque couche. Le détail des mesures et leur emplacement dans le code figurent au [DCT §6](#) ; la couverture par tests figure au [Cahier de recettes](#).

5.1 Authentification & autorisation

- **Clients** : e-mail + mot de passe (bcrypt via `password_hash`, coût ≥ 10), validation d'inscription par e-mail (jeton à usage unique, 24 h).
- **Administrateurs** : mot de passe + **2FA TOTP obligatoire** (RFC 6238) — implémentation maison côté PHP (`Core/Totp`), `googleauth` côté Java.
- **Contrôle d'accès** : middlewares `Authenticate` et `RequireAdmin` ; séparation stricte front-office / back-office.
- **Sessions** : cookies `HttpOnly` + `Secure` + `SameSite=Lax`, régénération de l'identifiant de session après connexion.

5.2 Chiffrement

- **En transit** : **HTTPS/TLS** obligatoire (Caddy + Let's Encrypt), redirection HTTP→HTTPS.
- **Au repos** : mots de passe **hachés** (bcrypt) ; données de carte **non stockées** (tokenisation Stripe) ; recommandation de chiffrement des archives de sauvegarde contenant des données personnelles.

5.3 Pare-feu & segmentation

- **OCI Security Lists** : seuls **443** (et 80 pour la redirection) sont ouverts ; **SSH (22)** restreint par IP ; **MySQL (3306) fermé au public**.
- **Segmentation applicative** : conteneurs sur un réseau bridge interne ; la base n'est jointe que par le conteneur web et, ponctuellement, via **tunnel SSH** pour l'application Java.
- **En-têtes de sécurité** : CSP, `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff` (`Core/Response`).

- **Protections applicatives** : anti-**injection SQL** (PDO préparé), anti-**XSS** (échappement systématique `e()`), anti-**CSRF** (jeton par session, `hash_equals`).

5.4 Conformité

- **RGPD** : pages CGU/confidentialité, politique de rétention, minimisation des données de paiement (aucun PAN stocké), droit à l'effacement applicable.
 - **PCI-DSS** : délégué à Stripe par tokenisation (aucune donnée de carte sur nos serveurs).
 - Objectif KPI : 100 % des failles cibles couvertes par un test documenté (voir Cahier de recettes).
-

6. DevOps & exploitation

6.1 CI/CD

- **Intégration continue** : **GitHub Actions** (`.github/workflows/ci.yml`) — à chaque push/pull request : lint PHP (`php -l`), **PHPUnit** (web), **JUnit** (Java), jobs parallèles bloquant la fusion en cas d'échec.
- **Déploiement** : **CD semi-automatisé** (`git pull` + `docker compose up -d --build` sur la VM), avec health check et procédure de rollback.
- Détail et évolution vers un CD entièrement automatisé (déploiement SSH sur tag) : PIPELINE_CICD.md.

6.2 Monitoring

- **KPI suivis** : temps de réponse (< 2 s), taux d'erreurs 5xx (< 1 %), disponibilité (> 99 %), requêtes SQL lentes, CPU/RAM de la VM.
- **Sonde de disponibilité** : contrôle externe (UptimeRobot / cron) sur un point de santé `GET /health`, alerte e-mail.
- **Ressources** : `docker stats`, `df` / `docker system df`.
- Piste d'évolution documentée : **Prometheus + Grafana** (métriques conteneurs) et **Sentry** (erreurs applicatives) — non

retenus au périmètre livré pour préserver l'empreinte minimale.
Détail : [SUPERVISION_LOGS.md](#).

6.3 Logs

- **Journalisation centralisée** via Docker (`stdout` / `stderr` de tous les conteneurs), rotation configurée.
- **Sources** : application PHP (`error_log` sur exceptions inattendues, page 500 générique en production), Apache, MySQL (slow query log), Caddy (JSON), script de sauvegarde.
- Une stack **ELK / Graylog** est une piste d'évolution (non nécessaire sur une VM unique).

6.4 Alerting & gestion des incidents

- **Alerting** : e-mail depuis la sonde de disponibilité (une escalade type PagerDuty/Opsgenie est surdimensionnée pour ce périmètre).
- **Suivi des bugs & incidents : GitHub Issues** (classification par sévérité S1-S4, lien issue ↔ PR ↔ correctif), registre d'incidents et post-mortem pour les incidents majeurs — voir [GESTION_INCIDENTS_BUGS.md](#).
- **Principe** : un bug corrigé = un test de non-régression ajouté.

7. Répartition rédactionnelle par candidat

Le DAT est un livrable **collectif unique**, chaque candidat étant responsable d'une partie (exigence Bloc 3).

Candidat	Parties du DAT
Adrien CACHOUX	§3 Infrastructure · §4 Architecture des données
Romain CANER	§2 Architecture applicative · §6 DevOps & exploitation
Ethan MIRBEAU	§5 Sécurité · §1 Vue d'ensemble

Répartition à adapter selon les tâches de développement réellement assurées par chacun (cf. backlog CDC §XVII).

8. Matrice de traçabilité des livrables Bloc 3

Exigence Bloc 3	Section DAT	Livrable détaillé
Déploiement & mise en production	§3	DEPLOIEMENT_ORACLE.md, INSTALLATION
Supervision & logs	§6.2-6.3	SUPERVISION_LOGS.md
Automatisation (CI/CD)	§6.1	PIPELINE_CICD.md
Sécurité applicative	§5	DCT §6, Cahier de recettes
Tests & performances	§2.4, §1.3	CAHIER_DE_RECETTES.md, PLAN_TESTS_PERFORMANCE.md
Gestion des incidents & bugs	§6.4	GESTION_INCIDENTS_BUGS.md
Bilan du projet	—	BILAN_PROJET.md
Documentation API	§2.4	api/openapi.yaml

9. Annexes & références

- [DCT — Dossier de Conception Technique](#)
- [Architecture applicative détaillée](#)
- [Guide d'installation](#)
- [Déploiement Oracle Cloud](#)
- [Pipeline CI/CD](#)
- [Supervision & logs](#)
- [Plan de tests de performance](#)
- [Gestion des incidents & bugs](#)
- [Infrastructure — sauvegardes & SSL](#)
- [Bilan du projet](#)
- [Documentation API \(OpenAPI\)](#)